

TPSM Viva Guide: Healthcare Experience Quality Analysis

This document is a viva-ready guide for the notebook `Healthcare\_Experience\_Quality\_Analysis.ipynb`.

It combines:

- cell-by-cell flow,
- code snippets used in the project,
- explanation of why each method was selected,
- expected viva questions and strong answers.

1) Project Overview (How to introduce in viva)

****Title:**** Patient Comfort and Healthcare Experience Analysis

****Core Goal:**** Analyze how comfort and service quality factors influence overall patient satisfaction (satisfaction in RM) and build predictive models for satisfaction.

****Research logic used in this project:****

1. Load and understand data
2. Clean and validate data types
3. Descriptive analysis (means, distributions, grouped summaries)
4. Inferential analysis (Spearman + Kruskal-Wallis)
5. Predictive analysis (Logistic Regression + Random Forest)
6. Model comparison, confusion matrix, feature importance
7. Final conclusions and recommendations

2) Dataset and Variables

Dataset file:

```
df = pd.read_csv("datasetsatisfaction.csv")
```

Target variable:

```
y = df["satisfaction in RM"]
```

Predictors:

- Check up appointment
- Time waiting
- Admin procedures

- Hygiene and cleaning
- Time of appointment
- Quality/experience dr.
- Specialists available
- Communication with dr
- Exact diagnosis
- Modern equipment
- friendly health care workers
- lab services
- availability of drugs
- waiting rooms
- hospital rooms quality
- parking, playing rooms, cafes

3) Import Section (What and why)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from scipy.stats import spearmanr, kruskal
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

import warnings
warnings.filterwarnings("ignore")

sns.set_style("whitegrid")
plt.rcParams["figure.figsize"] = (10,6)
```

Why these libraries?

- `pandas`, `numpy`: data preparation and tabular operations.
- `seaborn`, `matplotlib`: visualization for interpretation.
- `spearmanr`, `kruskal`: non-parametric inferential testing suitable for ordinal rating data.
- `sklearn`: machine learning workflow and evaluation.

4) Data Understanding Cells

Code

```
df.head()
```

```
df.info()
df.describe()
df.isnull().sum()
```

Explanation

- ``head()``: checks first rows and schema sanity.
- ``info()``: verifies row count, data types, non-null values.
- ``describe()``: summarizes central tendency and spread.
- ``isnull().sum()``: checks missingness column-wise.

Viva answer

"I validated structure and quality before analysis to ensure statistical tests and ML models receive clean numeric input."

5) Data Cleaning Cells

Code

```
df = df.dropna()

for col in df.columns:
    df[col] = pd.to_numeric(df[col], errors="coerce")

df = df.dropna()
```

Explanation

- First ``dropna()`` removes explicitly missing rows.
- ``to_numeric(..., errors="coerce")`` converts invalid entries to ``NaN``.
- Second ``dropna()`` removes rows that became invalid after coercion.

Why this method?

- Practical and robust for survey/rating datasets.
- Ensures inferential and predictive algorithms run on consistent numeric data.

Why not imputation here?

- Imputation can be used, but introduces assumptions.
- For this academic scope, transparent row removal is easier to justify.

6) Descriptive Analysis Cells

Mean ranking

```
mean_values = df.mean().sort_values(ascending=False)
```

```
mean_values
```

Plot mean values

```
sns.barplot(x=mean_values.index, y=mean_values.values)
plt.title("Average Patient Comfort Ratings")
plt.ylabel("Average Rating")
plt.xticks(rotation=75)
plt.show()
```

Distribution of target

```
sns.countplot(x="satisfaction in RM", data=df)
plt.title("Distribution of Healthcare Experience Quality")
plt.show()
```

Why these plots?

- Bar plot: best for comparing category averages.
- Countplot: reveals class distribution and imbalance risk.

7) Correlation Analysis Cells

Code

```
corr_matrix = df.corr(method="spearman")

plt.figure(figsize=(14,10))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Spearman Correlation Heatmap")
plt.show()

corr_with_satisfaction = corr_matrix["satisfaction in RM"].sort_values(ascending=False)
corr_with_satisfaction
```

Why Spearman and not Pearson?

- Data are ordinal ratings (Likert-type).
- Spearman captures monotonic associations with fewer strict assumptions.
- Pearson assumes linear relationships and interval-like behavior.

8) Grouping by Satisfaction Level

Code

```
df["satisfaction_group"] = df["satisfaction in RM"].map({
    1: "Low",
    2: "Medium",
    3: "High"
})
```

```
df["satisfaction_group"].value_counts()

group_means = df.groupby("satisfaction_group").mean()
group_means
```

Why map into groups?

- Improves interpretability for tables and comparative visuals.
- Useful for explaining subgroup behavior in viva.

9) Inferential Analysis (Main statistical evidence)

Hypothesis setup

- H0: Comfort-related factors have no significant relationship with overall satisfaction.
- H1: At least one comfort-related factor has a significant relationship with overall satisfaction.
- Decision rule: $\alpha = 0.05$

Spearman test for all factors

```
target_col = "satisfaction in RM"
exclude_cols = {target_col, "satisfaction_group"}
comfort_cols = [col for col in df.columns if col not in exclude_cols]

spearman_results = []
for col in comfort_cols:
    rho, p_value = spearmanr(df[col], df[target_col])
    spearman_results.append({
        "factor": col,
        "spearman_rho": rho,
        "p_value": p_value,
        "significant": p_value < 0.05
    })

spearman_df = pd.DataFrame(spearman_results).sort_values("p_value")
```

Kruskal-Wallis test for all factors

```
kw_results = []
for col in comfort_cols:
    g1 = df[df[target_col] == 1][col]
    g2 = df[df[target_col] == 2][col]
    g3 = df[df[target_col] == 3][col]
    stat, p_value = kruskal(g1, g2, g3)
    kw_results.append({
        "factor": col,
        "kw_statistic": stat,
        "p_value": p_value,
        "significant": p_value < 0.05
    })

kw_df = pd.DataFrame(kw_results).sort_values("p_value")
```

Project finding from both tests

Significant factors identified in both:

- Quality/experience dr.
- Exact diagnosis
- Modern equipment

Why Kruskal-Wallis instead of ANOVA?

- Non-parametric test suitable for ordinal/non-normal data.
- Compares more than two independent groups (Low/Medium/High).

10) Predictive Analysis Cells

Define feature matrix and target

```
X = df.drop(columns=["satisfaction in RM", "satisfaction_group"])
y = df["satisfaction in RM"]
```

Train-test split

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Why stratify?

- Preserves class ratio in train/test.
- Important because low-satisfaction class is very small.

11) Logistic Regression Cell

```
model1 = LogisticRegression(max_iter=1000)
model1.fit(X_train, y_train)
pred1 = model1.predict(X_test)

print("Logistic Regression Accuracy:", accuracy_score(y_test, pred1))
print(classification_report(y_test, pred1))
```

Why Logistic Regression?

- Baseline interpretable classifier.
- Good first benchmark before more complex models.

12) Random Forest Cell

```
model2 = RandomForestClassifier()
model2.fit(X_train, y_train)
pred2 = model2.predict(X_test)

print("Random Forest Accuracy:", accuracy_score(y_test, pred2))
print(classification_report(y_test, pred2))
```

Why Random Forest?

- Captures nonlinear relationships and feature interactions.
- Robust for tabular datasets and less sensitive to scaling.
- Provides feature importance for decision support.

13) Model Comparison Cells

Accuracy table

```
model_results = pd.DataFrame({
    "Model": ["Logistic Regression", "Random Forest"],
    "Accuracy": [accuracy_score(y_test, pred1), accuracy_score(y_test, pred2)]
})
```

Extended metric table

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

results = pd.DataFrame({
    "Model": ["Logistic Regression", "Random Forest"],
    "Accuracy": [
        accuracy_score(y_test, pred1),
        accuracy_score(y_test, pred2)
    ],
    "Precision (weighted)": [
        precision_score(y_test, pred1, average="weighted", zero_division=0),
        precision_score(y_test, pred2, average="weighted", zero_division=0)
    ],
    "Recall (weighted)": [
        recall_score(y_test, pred1, average="weighted", zero_division=0),
        recall_score(y_test, pred2, average="weighted", zero_division=0)
    ],
    "F1-score (weighted)": [
        f1_score(y_test, pred1, average="weighted", zero_division=0),
        f1_score(y_test, pred2, average="weighted", zero_division=0)
    ]
})
```

Why more than accuracy?

- Accuracy alone is weak under class imbalance.
- Precision/Recall/F1 provide class-sensitive performance context.

14) Confusion Matrix and Feature Importance Cells

Confusion matrix

```
cm = confusion_matrix(y_test, pred2)
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.title("Confusion Matrix")
plt.show()
```

Feature importance

```
importance_df = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model2.feature_importances_
}).sort_values("Importance", ascending=False)
```

Why these?

- Confusion matrix: class-level error diagnosis.
- Feature importance: which variables matter most for prediction and policy.

15) Final Conclusion (Viva-ready)

1. Inferential tests (Spearman + Kruskal-Wallis) reject global H0 at $\alpha = 0.05$.
2. The most consistent significant factors are:
 - Quality/experience dr.
 - Exact diagnosis
 - Modern equipment
3. Random Forest outperformed Logistic Regression in predictive accuracy.
4. Statistical and ML results align, increasing confidence in findings.

16) High-Probability Viva Questions and Answers

Q1: Why Spearman and not Pearson?

Because the variables are ordinal ratings and Spearman is a non-parametric monotonic association test with fewer assumptions.

Q2: Why Kruskal-Wallis and not ANOVA?

Kruskal-Wallis is non-parametric and better suited for ordinal/non-normal data with 3 independent groups.

Q3: Why use two models?

Logistic Regression gives an interpretable baseline; Random Forest captures nonlinear patterns and interactions.

Q4: Why stratified split?

To preserve class proportions in train and test sets due to minority low-satisfaction class.

Q5: Main limitation?

Class imbalance (very few low-class samples), no cross-validation/hyperparameter tuning, and potential survey-scale constraints.

17) 60-second Final Viva Script

"This project investigates whether healthcare comfort and service quality factors affect overall patient satisfaction. I started with data understanding and cleaning using missing-value handling and numeric coercion to ensure data quality. I performed descriptive analysis and then inferential tests using Spearman correlation and Kruskal-Wallis because the data are ordinal ratings and non-parametric methods are more appropriate than Pearson and ANOVA. Both tests identified key significant factors, especially doctor quality/experience, exact diagnosis, and modern equipment, so the global null hypothesis was rejected at alpha 0.05. For predictive analysis, I used Logistic Regression as a baseline and Random Forest as a nonlinear model with stratified train-test split. Random Forest achieved better performance and feature importance aligned with inferential findings. Therefore, the study concludes that selected healthcare quality dimensions significantly influence patient satisfaction and can be used to predict satisfaction outcomes."